isabelle-purepy

nhuber

October 16, 2026

Contents

theory *Syntax*
  imports *Main*
begin

type-alias *var* = *string*

type-synonym *qualified-name* = *var* × *var list*

datatype *lit* =
    LitNumber int
  | *LitString string*
  | *LitBool bool*
  | *LitNone*

datatype *pat* =
    PatLiteral lit
  | *PatNegLit int*
  | *PatVar var*
  | *PatWildcard*
  | *PatList pat list*
  | *PatTuple pat list*
  | *PatConstr qualified-name pat list* (*var* × *pat*) *list*
  | *PatDict* (*var* × *pat*) *list*
  | *PatAs pat var*

datatype *bop* =
    BOpEq

| *BOpNeq*
| *BOpLt*
| *BOpLeq*
| *BOpGt*
| *BOpGeq*
| *BOpBitOr*
| *BOpAdd*
| *BOpSub*
| *BOpMul*
| *BOpDiv*

datatype *uop* = *UOpNot* | *UOpNeg*

datatype *expr* =
  ExprVar var
| *ExprLiteral lit*
| *ExprCall expr expr list* (*var* × *expr*) *list*
| *ExprDict* (*expr* × *expr*) *list*
| *ExprAttribute expr var*
| *ExprSubscript expr expr list*
| *ExprBinOp bop expr expr*
| *ExprUnOp uop expr*
| *ExprAnd expr expr*
| *ExprOr expr expr*
| *ExprCond expr expr expr*
| *ExprList expr list*
| *ExprTuple expr list*
| *ExprListComp expr guard list*
| *ExprDictComp* (*expr* × *expr*) *guard list*
| *ExprLambda var list expr*

and *guard* =
  Guard expr
| *Generator var expr*

datatype *ty* =
  TypeAny
| *TypeNone*
| *TypeBool*
| *TypeInt*
| *TypeFloat*

| *TypeStr*
| *TypeList ty*
| *TypeTuple ty list*
| *TypeDict ty*
| *TypeCallable ty list ty*
| *TypeClass var*
| *TypeUnion ty ty*

datatype *import* =
  Import qualified-name
| *FromImport qualified-name var list*

datatype *stmt* =
  StmtPass
| *StmtSeq stmt stmt*
| *StmtAssign var ty option expr*
| *StmtExpr expr*
| *StmtAssert expr*
| *StmtAssertMsg expr expr*
| *StmtReturnNone*
| *StmtReturn expr*
| *StmtIf expr stmt* (*expr* × *stmt*) *list stmt option*
| *StmtMatch expr* (*pat* × *stmt*) *list*
| *StmtMutualRegion func list*

and *func* = *Def var* (*var* × *ty*) *list ty stmt*

datatype *fields* =
  FieldList (*string* × *ty*) *list*
| *EmptyFieldList*

datatype *top-stmt* =
  Stmt stmt
| *TopSeq top-stmt top-stmt*
| *Dataclass var fields*
| *DataclassExtend var var fields*

datatype *module-body* = *ModuleBody import list top-stmt*

end
theory *Semantics*
  imports *Syntax HOL.Map*
begin

no-notation (*ASCII*) *comp* (**infixl** ‹*o*› *55*)

datatype *val* =
    VLit lit
  | *VList val list*
  | *VTuple val list*
  | *VLamClosure var* ⇀ *val var list expr* (*Lam′*(-, -, -′))
  | *VDefClosure var* ⇀ *val func list nat* (*Def′*(-, -, -′))
  | *VUnbound*

type-synonym *env* = *var* ⇀ *val*
abbreviation *ε* :: *env* **where** *ε* ≡ *Map.empty*

datatype *termination-kind* =
    TypeError
  | *IndexError*
  | *KeyError*
  | *ZeroDivisionError*
  | *AssertionError val option*
  | *AttributeError*
  | *SystemExit int*
  | *Undef*

datatype *'a result* = *Val 'a* | *Aborts termination-kind*

datatype *outcome* = *Assigns env* | *Returns val* | *Aborts*$_o$ *termination-kind*

consts *mlet* :: *'a result* ⇒ (*'a* ⇒ *'b*) ⇒ *'b*

fun *mlet-result* :: *'a result* ⇒ (*'a* ⇒ *'b result*) ⇒ *'b result*
where
mlet-result (*Val v*) *f* = *f v* |
mlet-result (*Aborts κ*) *f* = *Aborts κ*

fun *mlet-outcome* :: *'a result* ⇒ (*'a* ⇒ *outcome*) ⇒ *outcome*
where
mlet-outcome (*Val v*) *f* = *f v* |

```
mlet-outcome (Aborts κ) f = Aborts_o κ

adhoc-overloading mlet ⇌ mlet-result mlet-outcome

fun compose :: outcome ⇒ outcome ⇒ outcome (infixl · 70)
where
Assigns ϱ · Assigns ϱ' = Assigns (ϱ ++ ϱ') |
Assigns ϱ · o = o |
Returns v · o = Returns v |
Aborts_o κ · o = Aborts_o κ

fun call-result :: outcome ⇒ val result
where
call-result (Returns v) = Val v |
call-result (Assigns ϱ) = Val (VLit LitNone) |
call-result (Aborts_o κ) = Aborts κ

fun is-closure :: val ⇒ bool
where
is-closure Lam(-, -, -) = True |
is-closure Def(-, -, -) = True |
is-closure - = False

fun eq :: val ⇒ val ⇒ bool result and eq-elems :: val list ⇒ val list ⇒ bool
result
where
eq (VLit (LitNumber n)) (VLit (LitNumber n')) = Val (n = n') |
eq (VLit (LitString w)) (VLit (LitString w')) = Val (w = w') |
eq (VLit (LitBool b)) (VLit (LitBool b')) = Val (b = b') |
eq (VLit LitNone) (VLit LitNone) = Val True |
eq (VLit (LitBool -)) (VLit (LitNumber -)) = Aborts Undef |
eq (VLit (LitNumber -)) (VLit (LitBool -)) = Aborts Undef |
eq (VList vs) (VList vs') = eq-elems vs vs' |
eq (VTuple vs) (VTuple vs') = eq-elems vs vs' |
eq Lam(-, -, -) Lam(-, -, -) = Aborts Undef |
eq Lam(-, -, -) Def(-, -, -) = Aborts Undef |
eq Def(-, -, -) Lam(-, -, -) = Aborts Undef |
eq Def(-, -, -) Def(-, -, -) = Aborts Undef |
eq - - = Val False |

eq-elems [] [] = Val True |
eq-elems (v # vs) (v' # vs') =
  (if length vs ≠ length vs' then Val False
   else if eq v v' = Val True then eq-elems vs vs' else eq v v') |
```

eq-elems - - = Val False

fun *binop-meaning* :: *bop* ⇒ *val* ⇒ *val* ⇒ *val result*
where
binop-meaning BOpEq v v′ = mlet (eq v v′) (λb. Val (VLit (LitBool b))) |
binop-meaning BOpNeq v v′ = mlet (eq v v′) (λb. Val (VLit (LitBool (¬ b)))) |
binop-meaning BOpLt (VLit (LitNumber n)) (VLit (LitNumber n′)) = Val (VLit (LitBool (n < n′))) |
binop-meaning BOpLeq (VLit (LitNumber n)) (VLit (LitNumber n′)) = Val (VLit (LitBool (n ≤ n′))) |
binop-meaning BOpGt (VLit (LitNumber n)) (VLit (LitNumber n′)) = Val (VLit (LitBool (n > n′))) |
binop-meaning BOpGeq (VLit (LitNumber n)) (VLit (LitNumber n′)) = Val (VLit (LitBool (n ≥ n′))) |
binop-meaning BOpAdd (VLit (LitNumber n)) (VLit (LitNumber n′)) = Val (VLit (LitNumber (n + n′))) |
binop-meaning BOpSub (VLit (LitNumber n)) (VLit (LitNumber n′)) = Val (VLit (LitNumber (n − n′))) |
binop-meaning BOpMul (VLit (LitNumber n)) (VLit (LitNumber n′)) = Val (VLit (LitNumber (n ∗ n′))) |
binop-meaning BOpDiv (VLit (LitNumber n)) (VLit (LitNumber n′)) =
(if n′ = 0 then Aborts ZeroDivisionError else Val (VLit (LitNumber (n div n′)))) |
binop-meaning - - - = Aborts TypeError

fun *unop-meaning* :: *uop* ⇒ *val* ⇒ *val result*
where
unop-meaning UOpNot (VLit (LitBool b)) = Val (VLit (LitBool (¬ b))) |
unop-meaning UOpNeg (VLit (LitNumber n)) = Val (VLit (LitNumber (− n))) |
unop-meaning - - = Aborts TypeError

fun *def-name* :: *func* ⇒ *var*
where
def-name (Def f - - -) = f

fun *binds-pat* :: *pat* ⇒ *var set*
where
binds-pat (PatVar x) = {x} |
binds-pat (PatAs p x) = binds-pat p ∪ {x} |
binds-pat (PatList ps) = ⋃ (set (map binds-pat ps)) |
binds-pat (PatTuple ps) = ⋃ (set (map binds-pat ps)) |
binds-pat (PatConstr q ps kps) = ⋃ (set (map binds-pat ps)) ∪ ⋃ (set (map

```
(binds-pat ∘ snd) kps)) |
binds-pat (PatDict kps) = ⋃ (set (map (binds-pat ∘ snd) kps)) |
binds-pat - = {}

fun binds :: stmt ⇒ var set
where
binds (StmtSeq s s′) = binds s ∪ binds s′ |
binds (StmtIf - s elifs else′) =
  binds s ∪ ⋃ (set (map (λ(-, s′). binds s′) elifs)) ∪ (case else′ of None ⇒
{} | Some s′ ⇒ binds s′) |
binds (StmtMatch - cases) = ⋃ (set (map (λ(p, s). binds-pat p ∪ binds s)
cases)) |
binds - = {}

fun assigns :: stmt ⇒ var set
where
assigns (StmtAssign x - -) = {x} |
assigns (StmtSeq s s′) = assigns s ∪ assigns s′ |
assigns (StmtMutualRegion ds) = set (map def-name ds) |
assigns (StmtIf - s elifs else′) =
  assigns s ∪ ⋃ (set (map (λ(-, s′). assigns s′) elifs)) ∪ (case else′ of None
⇒ {} | Some s′ ⇒ assigns s′) |
assigns (StmtMatch - cases) = ⋃ (set (map (λ(p, s). binds-pat p ∪ assigns
s) cases)) |
assigns - = {}

fun declares :: stmt ⇒ var list
where
declares (StmtAssign x (Some -) -) = [x] |
declares (StmtSeq s s′) = declares s @ declares s′ |
declares (StmtMutualRegion ds) = map def-name ds |
declares (StmtIf - s elifs else′) =
  declares s @ concat (map (λ(-, s′). declares s′) elifs) @ (case else′ of None
⇒ [] | Some s′ ⇒ declares s′) |
declares (StmtMatch - cases) = concat (map (λ(-, s). declares s) cases) |
declares - = []

definition scope :: var set ⇒ stmt ⇒ env option
where
scope Y s =
  (if distinct (declares s) ∧ Y ∩ set (declares s) = {} ∧ Y ∩ binds s = {}
∧ set (declares s) ∩ binds s = {}
   then Some (λx. if x ∈ assigns s − Y then Some VUnbound else None)
   else None)
```

abbreviation *true* :: *val* **where** *true* ≡ *VLit* (*LitBool True*)
abbreviation *false* :: *val* **where** *false* ≡ *VLit* (*LitBool False*)

type-synonym *elif-block* = (*expr* × *stmt*) *list* × *stmt option*

inductive
eval-expr :: *env* ⇒ *expr* ⇒ *val result* ⇒ *bool* (-, - ⇒$_e$ - [60,0,60] 60) **and**
eval-exprs :: *env* ⇒ *expr list* ⇒ *val list result* ⇒ *bool* (-, - ⇒$_{es}$ - [60,0,60] 60) **and**
eval-stmt :: *env* ⇒ *stmt* ⇒ *outcome* ⇒ *bool* (-, - ⇒$_s$ - [60,0,60] 60) **and**
eval-elif :: *env* ⇒ *elif-block* ⇒ *outcome* ⇒ *bool* (-, - ⇒$_{elif}$ - [60,0,60] 60)
where
EvalVar: ϱ *x* = *Some v* ⟹ ϱ, *ExprVar x* ⇒$_e$ *Val v* |
EvalLiteral: ϱ, *ExprLiteral l* ⇒$_e$ *Val* (*VLit l*) |
EvalLambda: ϱ, *ExprLambda xs e* ⇒$_e$ *Val Lam*(ϱ, *xs*, *e*) |
EvalUnOp: ϱ, *e* ⇒$_e$ *r* ⟹ ϱ, *ExprUnOp op e* ⇒$_e$ *mlet r* (*unop-meaning op*) |
EvalBinOp: ⟦ ϱ, *e* ⇒$_e$ *Val v* ; ϱ, *e′* ⇒$_e$ *r* ⟧
⟹ ϱ, *ExprBinOp op e e′* ⇒$_e$ *mlet r* (*binop-meaning op v*) |
EvalBinOpFail: ϱ, *e* ⇒$_e$ *Aborts* κ ⟹ ϱ, *ExprBinOp op e e′* ⇒$_e$ *Aborts* κ |
EvalAndFalse: ϱ, *e* ⇒$_e$ *Val false* ⟹ ϱ, *ExprAnd e e′* ⇒$_e$ *Val false* |
EvalAndTrue: ⟦ ϱ, *e* ⇒$_e$ *Val true* ; ϱ, *e′* ⇒$_e$ *r* ⟧ ⟹ ϱ, *ExprAnd e e′* ⇒$_e$ *r* |
EvalAndFail: ϱ, *e* ⇒$_e$ *Aborts* κ ⟹ ϱ, *ExprAnd e e′* ⇒$_e$ *Aborts* κ |
EvalOrTrue: ϱ, *e* ⇒$_e$ *Val true* ⟹ ϱ, *ExprOr e e′* ⇒$_e$ *Val true* |
EvalOrFalse: ⟦ ϱ, *e* ⇒$_e$ *Val false* ; ϱ, *e′* ⇒$_e$ *r* ⟧ ⟹ ϱ, *ExprOr e e′* ⇒$_e$ *r* |
EvalOrFail: ϱ, *e* ⇒$_e$ *Aborts* κ ⟹ ϱ, *ExprOr e e′* ⇒$_e$ *Aborts* κ |
EvalCondTrue: ⟦ ϱ, *e* ⇒$_e$ *Val true* ; ϱ, *e1* ⇒$_e$ *r* ⟧ ⟹ ϱ, *ExprCond e1 e e2* ⇒$_e$ *r* |
EvalCondFalse: ⟦ ϱ, *e* ⇒$_e$ *Val false* ; ϱ, *e2* ⇒$_e$ *r* ⟧ ⟹ ϱ, *ExprCond e1 e e2* ⇒$_e$ *r* |
EvalCondFail: ϱ, *e* ⇒$_e$ *Aborts* κ ⟹ ϱ, *ExprCond e1 e e2* ⇒$_e$ *Aborts* κ |
EvalList: ϱ, *es* ⇒$_{es}$ *Val vs* ⟹ ϱ, *ExprList es* ⇒$_e$ *Val* (*VList vs*) |
EvalListFail: ϱ, *es* ⇒$_{es}$ *Aborts* κ ⟹ ϱ, *ExprList es* ⇒$_e$ *Aborts* κ |
EvalTuple: ϱ, *es* ⇒$_{es}$ *Val vs* ⟹ ϱ, *ExprTuple es* ⇒$_e$ *Val* (*VTuple vs*) |
EvalTupleFail: ϱ, *es* ⇒$_{es}$ *Aborts* κ ⟹ ϱ, *ExprTuple es* ⇒$_e$ *Aborts* κ |
EvalCallLambda:
⟦ ϱ, *e* ⇒$_e$ *Val Lam*(ϱ′, *xs*, *e′*)
; ϱ, *es* ⇒$_{es}$ *Val us*
; *length xs* = *length us*
; ϱ′(*xs* [↦] *us*), *e′* ⇒$_e$ *r*

$\rrbracket \Longrightarrow \varrho,\ \mathit{ExprCall}\ e\ es\ []\ \Rightarrow_e\ r\ |$
EvalCallLambdaArity:
$[\![\ \varrho,\ e\ \Rightarrow_e\ \mathit{Val}\ \mathit{Lam}(\varrho',\ xs,\ e')$
$;\ \varrho,\ es\ \Rightarrow_{es}\ \mathit{Val}\ us$
$;\ \mathit{length}\ xs \neq \mathit{length}\ us$
$\rrbracket \Longrightarrow \varrho,\ \mathit{ExprCall}\ e\ es\ []\ \Rightarrow_e\ \mathit{Aborts}\ \mathit{TypeError}\ |$
EvalCallDef:
$[\![\ \varrho,\ e\ \Rightarrow_e\ \mathit{Val}\ \mathit{Def}(\varrho',\ ds,\ j)$
$;\ ds\ !\ j = \mathit{Def}\ f\ xps\ \psi\ s$
$;\ vs = \mathit{map}\ (\lambda i.\ \mathit{Def}(\varrho',\ ds,\ i))\ [0..<\mathit{length}\ ds]$
$;\ \varrho,\ es\ \Rightarrow_{es}\ \mathit{Val}\ us$
$;\ \mathit{length}\ xps = \mathit{length}\ us$
$;\ \mathit{scope}\ (\mathit{set}\ (\mathit{map}\ \mathit{fst}\ xps))\ s = \mathit{Some}\ \varrho s$
$;\ \varrho'(\mathit{map}\ \mathit{def\text{-}name}\ ds\ [\mapsto]\ vs,\ \mathit{map}\ \mathit{fst}\ xps\ [\mapsto]\ us)\ +\!\!+\ \varrho s,\ s\ \Rightarrow_s\ o$
$\rrbracket \Longrightarrow \varrho,\ \mathit{ExprCall}\ e\ es\ []\ \Rightarrow_e\ \mathit{call\text{-}result}\ o\ |$
EvalCallDefArity:
$[\![\ \varrho,\ e\ \Rightarrow_e\ \mathit{Val}\ \mathit{Def}(\varrho',\ ds,\ j)$
$;\ ds\ !\ j = \mathit{Def}\ f\ xps\ \psi\ s$
$;\ \varrho,\ es\ \Rightarrow_{es}\ \mathit{Val}\ us$
$;\ \mathit{length}\ xps \neq \mathit{length}\ us$
$\rrbracket \Longrightarrow \varrho,\ \mathit{ExprCall}\ e\ es\ []\ \Rightarrow_e\ \mathit{Aborts}\ \mathit{TypeError}\ |$
EvalCallNonFun:
$[\![\ \varrho,\ e\ \Rightarrow_e\ \mathit{Val}\ v\ ;\ \varrho,\ es\ \Rightarrow_{es}\ \mathit{Val}\ us\ ;\ \neg\ \mathit{is\text{-}closure}\ v\ ]\!]$
$\Longrightarrow \varrho,\ \mathit{ExprCall}\ e\ es\ []\ \Rightarrow_e\ \mathit{Aborts}\ \mathit{TypeError}\ |$
EvalCallFail: $\varrho,\ e\ \Rightarrow_e\ \mathit{Aborts}\ \kappa \Longrightarrow \varrho,\ \mathit{ExprCall}\ e\ es\ []\ \Rightarrow_e\ \mathit{Aborts}\ \kappa\ |$
EvalCallFailArgs:
$[\![\ \varrho,\ e\ \Rightarrow_e\ \mathit{Val}\ v\ ;\ \varrho,\ es\ \Rightarrow_{es}\ \mathit{Aborts}\ \kappa\ ]\!] \Longrightarrow \varrho,\ \mathit{ExprCall}\ e\ es\ []\ \Rightarrow_e\ \mathit{Aborts}\ \kappa\ |$

EvalExprsNil: $\varrho,\ []\ \Rightarrow_{es}\ \mathit{Val}\ []\ |$
EvalExprsCons: $[\![\ \varrho,\ e\ \Rightarrow_e\ \mathit{Val}\ v\ ;\ \varrho,\ es\ \Rightarrow_{es}\ r\ ]\!]$
$\Longrightarrow \varrho,\ e\ \#\ es\ \Rightarrow_{es}\ \mathit{mlet}\ r\ (\lambda vs.\ \mathit{Val}\ (v\ \#\ vs))\ |$
EvalExprsFail: $\varrho,\ e\ \Rightarrow_e\ \mathit{Aborts}\ \kappa \Longrightarrow \varrho,\ e\ \#\ es\ \Rightarrow_{es}\ \mathit{Aborts}\ \kappa\ |$

EvalPass: $\varrho,\ \mathit{StmtPass}\ \Rightarrow_s\ \mathit{Assigns}\ \varepsilon\ |$
EvalAssign: $\varrho,\ e\ \Rightarrow_e\ r \Longrightarrow \varrho,\ \mathit{StmtAssign}\ x\ \pi\ e\ \Rightarrow_s\ \mathit{mlet}\ r\ (\lambda v.\ \mathit{Assigns}\ [x\ \mapsto\ v])\ |$
EvalExprStmt: $\varrho,\ e\ \Rightarrow_e\ r \Longrightarrow \varrho,\ \mathit{StmtExpr}\ e\ \Rightarrow_s\ \mathit{mlet}\ r\ (\lambda v.\ \mathit{Assigns}\ \varepsilon)\ |$
EvalReturn: $\varrho,\ e\ \Rightarrow_e\ r \Longrightarrow \varrho,\ \mathit{StmtReturn}\ e\ \Rightarrow_s\ \mathit{mlet}\ r\ (\lambda v.\ \mathit{Returns}\ v)\ |$
EvalReturnNone: $\varrho,\ \mathit{StmtReturnNone}\ \Rightarrow_s\ \mathit{Returns}\ (\mathit{VLit}\ \mathit{LitNone})\ |$
EvalDefs:
$vs = \mathit{map}\ (\lambda i.\ \mathit{Def}(\varrho,\ ds,\ i))\ [0..<\mathit{length}\ ds]$
$\Longrightarrow \varrho,\ \mathit{StmtMutualRegion}\ ds\ \Rightarrow_s\ \mathit{Assigns}\ (\varepsilon(\mathit{map}\ \mathit{def\text{-}name}\ ds\ [\mapsto]\ vs))\ |$
EvalAssert: $\varrho,\ e\ \Rightarrow_e\ \mathit{Val}\ \mathit{true} \Longrightarrow \varrho,\ \mathit{StmtAssert}\ e\ \Rightarrow_s\ \mathit{Assigns}\ \varepsilon\ |$

EvalAssertFalse: $\varrho,\ e \Rightarrow_e \mathit{Val}\ \mathit{false} \Longrightarrow \varrho,\ \mathit{StmtAssert}\ e \Rightarrow_s \mathit{Aborts}_o\ (\mathit{AssertionError}\ \mathit{None})$ |
EvalAssertFail: $\varrho,\ e \Rightarrow_e \mathit{Aborts}\ \kappa \Longrightarrow \varrho,\ \mathit{StmtAssert}\ e \Rightarrow_s \mathit{Aborts}_o\ \kappa$ |
EvalAssertMsg: $\varrho,\ e \Rightarrow_e \mathit{Val}\ \mathit{true} \Longrightarrow \varrho,\ \mathit{StmtAssertMsg}\ e\ e' \Rightarrow_s \mathit{Assigns}\ \varepsilon$ |
EvalAssertMsgFalse: $[\![\ \varrho,\ e \Rightarrow_e \mathit{Val}\ \mathit{false}\ ;\ \varrho,\ e' \Rightarrow_e r\ ]\!]$
$\Longrightarrow \varrho,\ \mathit{StmtAssertMsg}\ e\ e' \Rightarrow_s \mathit{mlet}\ r\ (\lambda w.\ \mathit{Aborts}_o\ (\mathit{AssertionError}\ (\mathit{Some}\ w)))$ |
EvalAssertMsgFail: $\varrho,\ e \Rightarrow_e \mathit{Aborts}\ \kappa \Longrightarrow \varrho,\ \mathit{StmtAssertMsg}\ e\ e' \Rightarrow_s \mathit{Aborts}_o\ \kappa$ |
EvalIfTrue: $[\![\ \varrho,\ e \Rightarrow_e \mathit{Val}\ \mathit{true}\ ;\ \varrho,\ s \Rightarrow_s o\ ]\!]$
$\Longrightarrow \varrho,\ \mathit{StmtIf}\ e\ s\ \mathit{elifs}\ \mathit{else}' \Rightarrow_s o$ |
EvalIfFalse: $[\![\ \varrho,\ e \Rightarrow_e \mathit{Val}\ \mathit{false}\ ;\ \varrho,\ (\mathit{elifs},\ \mathit{else}') \Rightarrow_{elif} o\ ]\!]$
$\Longrightarrow \varrho,\ \mathit{StmtIf}\ e\ s\ \mathit{elifs}\ \mathit{else}' \Rightarrow_s o$ |
EvalIfFail: $\varrho,\ e \Rightarrow_e \mathit{Aborts}\ \kappa \Longrightarrow \varrho,\ \mathit{StmtIf}\ e\ s\ \mathit{elifs}\ \mathit{else}' \Rightarrow_s \mathit{Aborts}_o\ \kappa$ |
EvalSeq: $[\![\ \varrho,\ s \Rightarrow_s \mathit{Assigns}\ \varrho'\ ;\ \varrho +\!\!+ \varrho',\ s' \Rightarrow_s o\ ]\!]$
$\Longrightarrow \varrho,\ \mathit{StmtSeq}\ s\ s' \Rightarrow_s \mathit{Assigns}\ \varrho' \bullet o$ |
EvalSeqReturn: $\varrho,\ s \Rightarrow_s \mathit{Returns}\ v \Longrightarrow \varrho,\ \mathit{StmtSeq}\ s\ s' \Rightarrow_s \mathit{Returns}\ v$ |
EvalSeqFail: $\varrho,\ s \Rightarrow_s \mathit{Aborts}_o\ \kappa \Longrightarrow \varrho,\ \mathit{StmtSeq}\ s\ s' \Rightarrow_s \mathit{Aborts}_o\ \kappa$ |

EvalIfNone: $\varrho,\ ([],\ \mathit{None}) \Rightarrow_{elif} \mathit{Assigns}\ \varepsilon$ |
EvalElse: $\varrho,\ s \Rightarrow_s o \Longrightarrow \varrho,\ ([],\ \mathit{Some}\ s) \Rightarrow_{elif} o$ |
EvalElifTrue: $[\![\ \varrho,\ e \Rightarrow_e \mathit{Val}\ \mathit{true}\ ;\ \varrho,\ s \Rightarrow_s o\ ]\!]$
$\Longrightarrow \varrho,\ ((e,\ s)\ \#\ \mathit{elifs},\ \mathit{else}') \Rightarrow_{elif} o$ |
EvalElifFalse: $[\![\ \varrho,\ e \Rightarrow_e \mathit{Val}\ \mathit{false}\ ;\ \varrho,\ (\mathit{elifs},\ \mathit{else}') \Rightarrow_{elif} o\ ]\!]$
$\Longrightarrow \varrho,\ ((e,\ s)\ \#\ \mathit{elifs},\ \mathit{else}') \Rightarrow_{elif} o$ |
EvalElifFail: $\varrho,\ e \Rightarrow_e \mathit{Aborts}\ \kappa \Longrightarrow \varrho,\ ((e,\ s)\ \#\ \mathit{elifs},\ \mathit{else}') \Rightarrow_{elif} \mathit{Aborts}_o\ \kappa$

lemmas *eval-induct* = *eval-expr-eval-exprs-eval-stmt-eval-elif.inducts*

end
theory *Eval*
  imports *Semantics*
begin

end
theory *Correctness*
  imports *Eval*
begin

inductive-cases *VarE*: $\varrho,\ \mathit{ExprVar}\ x \Rightarrow_e r$
inductive-cases *LiteralE*: $\varrho,\ \mathit{ExprLiteral}\ l \Rightarrow_e r$
inductive-cases *LambdaE*: $\varrho,\ \mathit{ExprLambda}\ \mathit{xs}\ e \Rightarrow_e r$

inductive-cases *UnOpE*: ϱ, *ExprUnOp op e* $\Rightarrow_e$ *r*
inductive-cases *BinOpE*: ϱ, *ExprBinOp op e e'* $\Rightarrow_e$ *r*
inductive-cases *AndE*: ϱ, *ExprAnd e e'* $\Rightarrow_e$ *r*
inductive-cases *OrE*: ϱ, *ExprOr e e'* $\Rightarrow_e$ *r*
inductive-cases *CondE*: ϱ, *ExprCond e1 e e2* $\Rightarrow_e$ *r*
inductive-cases *ListE*: ϱ, *ExprList es* $\Rightarrow_e$ *r*
inductive-cases *TupleE*: ϱ, *ExprTuple es* $\Rightarrow_e$ *r*
inductive-cases
CallE[*consumes 1*, *case-names CallLambda CallLambdaArity CallDef CallDefArity CallNonFun CallFail CallFailArgs*]:
ϱ, *ExprCall e es* [] $\Rightarrow_e$ *r*
inductive-cases *ExprsNilE*: ϱ, [] $\Rightarrow_{es}$ *rs*
inductive-cases *ExprsConsE*: ϱ, *e* # *es* $\Rightarrow_{es}$ *rs*
inductive-cases *PassE*: ϱ, *StmtPass* $\Rightarrow_s$ *o*
inductive-cases *AssignE*: ϱ, *StmtAssign x* π *e* $\Rightarrow_s$ *o*
inductive-cases *ExprStmtE*: ϱ, *StmtExpr e* $\Rightarrow_s$ *o*
inductive-cases *ReturnE*: ϱ, *StmtReturn e* $\Rightarrow_s$ *o*
inductive-cases *ReturnNoneE*: ϱ, *StmtReturnNone* $\Rightarrow_s$ *o*
inductive-cases *DefsE*: ϱ, *StmtMutualRegion ds* $\Rightarrow_s$ *o*
inductive-cases *AssertE*: ϱ, *StmtAssert e* $\Rightarrow_s$ *o*
inductive-cases *AssertMsgE*: ϱ, *StmtAssertMsg e e'* $\Rightarrow_s$ *o*
inductive-cases *IfE*: ϱ, *StmtIf e s elifs else'* $\Rightarrow_s$ *o*
inductive-cases *SeqE*: ϱ, *StmtSeq s s'* $\Rightarrow_s$ *o*
inductive-cases *ElifNilE*: ϱ, ([], *else'*) $\Rightarrow_{elif}$ *o*
inductive-cases *ElifConsE*: ϱ, ((*e*, *s*) # *elifs*, *else'*) $\Rightarrow_{elif}$ *o*

lemma *determinism*:
shows
expr-det: ϱ, *e* $\Rightarrow_e$ *r* $\Longrightarrow$ ϱ, *e* $\Rightarrow_e$ *r'* $\Longrightarrow$ *r* = *r'*
and *exprs-det*: ϱ, *es* $\Rightarrow_{es}$ *rs* $\Longrightarrow$ ϱ, *es* $\Rightarrow_{es}$ *rs'* $\Longrightarrow$ *rs* = *rs'*
and *stmt-det*: ϱ, *s* $\Rightarrow_s$ *o* $\Longrightarrow$ ϱ, *s* $\Rightarrow_s$ *o'* $\Longrightarrow$ *o* = *o'*
and *elif-det*: ϱ, *eb* $\Rightarrow_{elif}$ *o'* $\Longrightarrow$ ϱ, *eb* $\Rightarrow_{elif}$ *o''* $\Longrightarrow$ *o'* = *o''*
proof (*induction arbitrary*: *r'* **and** *rs'* **and** *o'* **and** *o''* *rule*: *eval-induct*)
case *EvalVar*
then show *?case* **by** (*auto elim*!: *VarE*)
next
case *EvalLiteral*
then show *?case* **by** (*auto elim*!: *LiteralE*)
next
case *EvalLambda*
then show *?case* **by** (*auto elim*!: *LambdaE*)
next
case *EvalUnOp*
from *EvalUnOp.prems EvalUnOp.hyps* **show** *?case*

```
    by (auto elim!: UnOpE dest: EvalUnOp.IH)
next
  case EvalBinOp
  from EvalBinOp.prems EvalBinOp.hyps show ?case
    by (auto elim!: BinOpE dest: EvalBinOp.IH)
next
  case EvalBinOpFail
  from EvalBinOpFail.prems EvalBinOpFail.hyps show ?case
    by (auto elim!: BinOpE dest: EvalBinOpFail.IH)
next
  case EvalAndFalse
  from EvalAndFalse.prems EvalAndFalse.hyps show ?case
    by (auto elim!: AndE dest: EvalAndFalse.IH)
next
  case EvalAndTrue
  from EvalAndTrue.prems EvalAndTrue.hyps show ?case
    by (auto elim!: AndE dest: EvalAndTrue.IH)
next
  case EvalAndFail
  from EvalAndFail.prems EvalAndFail.hyps show ?case
    by (auto elim!: AndE dest: EvalAndFail.IH)
next
  case EvalOrTrue
  from EvalOrTrue.prems EvalOrTrue.hyps show ?case
    by (auto elim!: OrE dest: EvalOrTrue.IH)
next
  case EvalOrFalse
  from EvalOrFalse.prems EvalOrFalse.hyps show ?case
    by (auto elim!: OrE dest: EvalOrFalse.IH)
next
  case EvalOrFail
  from EvalOrFail.prems EvalOrFail.hyps show ?case
    by (auto elim!: OrE dest: EvalOrFail.IH)
next
  case EvalCondTrue
  from EvalCondTrue.prems EvalCondTrue.hyps show ?case
    by (auto elim!: CondE dest: EvalCondTrue.IH)
next
  case EvalCondFalse
  from EvalCondFalse.prems EvalCondFalse.hyps show ?case
    by (auto elim!: CondE dest: EvalCondFalse.IH)
next
  case EvalCondFail
  from EvalCondFail.prems EvalCondFail.hyps show ?case
```

```
    by (auto elim!: CondE dest: EvalCondFail.IH)
next
  case EvalList
  from EvalList.prems EvalList.hyps show ?case
    by (auto elim!: ListE dest: EvalList.IH)
next
  case EvalListFail
  from EvalListFail.prems EvalListFail.hyps show ?case
    by (auto elim!: ListE dest: EvalListFail.IH)
next
  case EvalTuple
  from EvalTuple.prems EvalTuple.hyps show ?case
    by (auto elim!: TupleE dest: EvalTuple.IH)
next
  case EvalTupleFail
  from EvalTupleFail.prems EvalTupleFail.hyps show ?case
    by (auto elim!: TupleE dest: EvalTupleFail.IH)
next
  case EvalCallLambda
  from EvalCallLambda.prems EvalCallLambda.hyps show ?case
    by (auto elim!: CallE dest: EvalCallLambda.IH)
next
  case EvalCallLambdaArity
  from EvalCallLambdaArity.prems EvalCallLambdaArity.hyps show ?case
    by (auto elim!: CallE dest: EvalCallLambdaArity.IH)
next
  case (EvalCallDef ϱ e ϱ' ds j f xps ψ s vs es us ϱs o)
  from EvalCallDef.prems show ?case
  proof (cases rule: CallE)
    case (CallDef ϱ'' ds' j' f' xps' ψ' s' us' ϱs' o')
    from EvalCallDef.IH(1)[OF CallDef(2)] have ϱ'' = ϱ' ds' = ds j' = j
by auto
    with EvalCallDef.hyps(2) CallDef(3) have xps' = xps s' = s by auto
    moreover from EvalCallDef.IH(2)[OF CallDef(4)] have us' = us by
simp
    moreover from calculation EvalCallDef.hyps(6) CallDef(6) have ϱs'
= ϱs by simp
    ultimately have o' = o
      using EvalCallDef.IH(3) CallDef(7) ‹ϱ'' = ϱ'› ‹ds' = ds› Eval-
CallDef.hyps(3) by simp
    with CallDef(1) show ?thesis by simp
  next
    case (CallDefArity ϱ'' ds' j' f' xps' ψ' s' us')
    from EvalCallDef.IH(1)[OF CallDefArity(2)] have ds' = ds j' = j by
```

```
auto
    with EvalCallDef.hyps(2) CallDefArity(3) have xps' = xps by auto
    moreover from EvalCallDef.IH(2)[OF CallDefArity(4)] have us' =
us by simp
    ultimately show ?thesis using CallDefArity(5) EvalCallDef.hyps(5)
by simp
  qed (use EvalCallDef.hyps in ‹auto dest: EvalCallDef.IH›)
next
  case (EvalCallDefArity ϱ e ϱ' ds j f xps ψ s es us)
  from EvalCallDefArity.prems show ?case
  proof (cases rule: CallE)
    case (CallDef ϱ'' ds' j' f' xps' ψ' s' us' ϱs' o')
    from EvalCallDefArity.IH(1)[OF CallDef(2)] have ds' = ds j' = j by
auto
    with EvalCallDefArity.hyps(2) CallDef(3) have xps' = xps by auto
    moreover from EvalCallDefArity.IH(2)[OF CallDef(4)] have us' =
us by simp
    ultimately show ?thesis using CallDef(5) EvalCallDefArity.hyps(4)
by simp
  qed (use EvalCallDefArity.hyps in ‹auto dest: EvalCallDefArity.IH›)
next
  case EvalCallNonFun
  from EvalCallNonFun.prems EvalCallNonFun.hyps show ?case
    by (auto elim!: CallE dest: EvalCallNonFun.IH)
next
  case EvalCallFail
  from EvalCallFail.prems EvalCallFail.hyps show ?case
    by (auto elim!: CallE dest: EvalCallFail.IH)
next
  case EvalCallFailArgs
  from EvalCallFailArgs.prems EvalCallFailArgs.hyps show ?case
    by (auto elim!: CallE dest: EvalCallFailArgs.IH)
next
  case EvalExprsNil
  then show ?case by (auto elim!: ExprsNilE)
next
  case EvalExprsCons
  from EvalExprsCons.prems EvalExprsCons.hyps show ?case
    by (auto elim!: ExprsConsE dest: EvalExprsCons.IH)
next
  case EvalExprsFail
  from EvalExprsFail.prems EvalExprsFail.hyps show ?case
    by (auto elim!: ExprsConsE dest: EvalExprsFail.IH)
next
```

```
  case EvalPass
  then show ?case by (auto elim!: PassE)
next
  case EvalAssign
  from EvalAssign.prems EvalAssign.hyps show ?case
    by (auto elim!: AssignE dest: EvalAssign.IH)
next
  case EvalExprStmt
  from EvalExprStmt.prems EvalExprStmt.hyps show ?case
    by (auto elim!: ExprStmtE dest: EvalExprStmt.IH)
next
  case EvalReturn
  from EvalReturn.prems EvalReturn.hyps show ?case
    by (auto elim!: ReturnE dest: EvalReturn.IH)
next
  case EvalReturnNone
  then show ?case by (auto elim!: ReturnNoneE)
next
  case EvalDefs
  then show ?case by (auto elim!: DefsE)
next
  case EvalAssert
  from EvalAssert.prems EvalAssert.hyps show ?case
    by (auto elim!: AssertE dest: EvalAssert.IH)
next
  case EvalAssertFalse
  from EvalAssertFalse.prems EvalAssertFalse.hyps show ?case
    by (auto elim!: AssertE dest: EvalAssertFalse.IH)
next
  case EvalAssertFail
  from EvalAssertFail.prems EvalAssertFail.hyps show ?case
    by (auto elim!: AssertE dest: EvalAssertFail.IH)
next
  case EvalAssertMsg
  from EvalAssertMsg.prems EvalAssertMsg.hyps show ?case
    by (auto elim!: AssertMsgE dest: EvalAssertMsg.IH)
next
  case EvalAssertMsgFalse
  from EvalAssertMsgFalse.prems EvalAssertMsgFalse.hyps show ?case
    by (auto elim!: AssertMsgE dest: EvalAssertMsgFalse.IH)
next
  case EvalAssertMsgFail
  from EvalAssertMsgFail.prems EvalAssertMsgFail.hyps show ?case
    by (auto elim!: AssertMsgE dest: EvalAssertMsgFail.IH)
```

next
case *EvalIfTrue*
from *EvalIfTrue.prems EvalIfTrue.hyps* **show** *?case*
by (*auto elim*!: *IfE dest*: *EvalIfTrue.IH*)
next
case *EvalIfFalse*
from *EvalIfFalse.prems EvalIfFalse.hyps* **show** *?case*
by (*auto elim*!: *IfE dest*: *EvalIfFalse.IH*)
next
case *EvalIfFail*
from *EvalIfFail.prems EvalIfFail.hyps* **show** *?case*
by (*auto elim*!: *IfE dest*: *EvalIfFail.IH*)
next
case *EvalSeq*
from *EvalSeq.prems EvalSeq.hyps* **show** *?case*
by (*auto elim*!: *SeqE dest*: *EvalSeq.IH*)
next
case *EvalSeqReturn*
from *EvalSeqReturn.prems EvalSeqReturn.hyps* **show** *?case*
by (*auto elim*!: *SeqE dest*: *EvalSeqReturn.IH*)
next
case *EvalSeqFail*
from *EvalSeqFail.prems EvalSeqFail.hyps* **show** *?case*
by (*auto elim*!: *SeqE dest*: *EvalSeqFail.IH*)
next
case *EvalIfNone*
then show *?case* **by** (*auto elim*!: *ElifNilE*)
next
case *EvalElse*
from *EvalElse.prems EvalElse.hyps* **show** *?case*
by (*auto elim*!: *ElifNilE dest*: *EvalElse.IH*)
next
case *EvalElifTrue*
from *EvalElifTrue.prems EvalElifTrue.hyps* **show** *?case*
by (*auto elim*!: *ElifConsE dest*: *EvalElifTrue.IH*)
next
case *EvalElifFalse*
from *EvalElifFalse.prems EvalElifFalse.hyps* **show** *?case*
by (*auto elim*!: *ElifConsE dest*: *EvalElifFalse.IH*)
next
case *EvalElifFail*
from *EvalElifFail.prems EvalElifFail.hyps* **show** *?case*
by (*auto elim*!: *ElifConsE dest*: *EvalElifFail.IH*)
qed

end
theory *Examples*
  imports *Semantics*
begin

definition *ex-add* :: *expr*
where *ex-add* = *ExprBinOp BOpAdd* (*ExprLiteral* (*LitNumber 10*)) (*ExprLiteral* (*LitNumber 20*))

lemma *ex-add-eval*: ε, *ex-add* $\Rightarrow_e$ *Val* (*VLit* (*LitNumber 30*))
proof −
  have ε, *ex-add* $\Rightarrow_e$ *mlet* (*Val* (*VLit* (*LitNumber 20*))) (*binop-meaning BOpAdd* (*VLit* (*LitNumber 10*)))
    unfolding *ex-add-def* **by** (*rule EvalBinOp*[*OF EvalLiteral EvalLiteral*])
  then show *?thesis* **by** *simp*
qed

definition *ex-lambda* :: *expr*
where *ex-lambda* =
    ExprCall
      (*ExprLambda* [$''x''$] (*ExprBinOp BOpAdd* (*ExprVar* $''x''$) (*ExprLiteral* (*LitNumber 1*))))
       [*ExprLiteral* (*LitNumber 2*)] []

lemma *ex-lambda-eval*: ε, *ex-lambda* $\Rightarrow_e$ *Val* (*VLit* (*LitNumber 3*))
proof −
  have *args*: ε, [*ExprLiteral* (*LitNumber 2*)] $\Rightarrow_{es}$ *Val* [*VLit* (*LitNumber 2*)]
    using *EvalExprsCons*[*OF EvalLiteral EvalExprsNil*] **by** *simp*
  let *?ϱ* = ε([$''x''$] [$\mapsto$] [*VLit* (*LitNumber 2*)])
  have *?ϱ*, *ExprVar* $''x''$ $\Rightarrow_e$ *Val* (*VLit* (*LitNumber 2*)) **by** (*rule EvalVar*) *simp*
 then have *?ϱ*, *ExprBinOp BOpAdd* (*ExprVar* $''x''$) (*ExprLiteral* (*LitNumber 1*))
     $\Rightarrow_e$ *mlet* (*Val* (*VLit* (*LitNumber 1*))) (*binop-meaning BOpAdd* (*VLit* (*LitNumber 2*)))
    by (*rule EvalBinOp*[*OF - EvalLiteral*])
  then have *body*:
     ?ϱ, *ExprBinOp BOpAdd* (*ExprVar* $''x''$) (*ExprLiteral* (*LitNumber 1*)) $\Rightarrow_e$ *Val* (*VLit* (*LitNumber 3*))
    by *simp*
  show *?thesis*
   unfolding *ex-lambda-def* **by** (*rule EvalCallLambda*[*OF EvalLambda args - body*]) *simp*

qed

end